

Universidad de las Fuerzas Armadas ESPE

Departamento: Ciencias de la Computación

Carrera: Ingeniería de Software

Asignatura: Análisis y Diseño de Software

Instructora del curso: Ing. Jenny A Ruiz R MIS

Apellidos y nombres de los estudiantes:

- Bryan Alexander Rosero Delgado
- Cámeron Victoria Díaz Caisaluiza
- Benjamin Alejandro Robalino

NRC: 36839

Fecha de realización: 05/11/2026

U1 Guia2

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso de aplicación: SPR — Módulo de Gestión de Tienda (Productos y Pedidos)

Elemento	Descripción
Asignatura	Análisis y Diseño de Software (NRC 30719)
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base del proyecto	SPR (Sistema de Pedidos y Reservas para Canchas Don Pepito) — Módulo de Gestión de Tienda
Operaciones clave	Ingresar Producto (GT-11), Modificar Producto (GT-12), Buscar Producto (GT-10) y Generar Reporte de Pedidos (GT-02)
Requisitos trazados	RF-007 (Registrar), RF-009 (Modificar), RF-010 (Consultar) y RF-044 (Reporte pedidos)
Referencias base	Guia_PlantUML_SPR_Gestion_Tienda.docx (Guía 1), 02_Guia_2_casos_de_uso_clases_y_secuencia_CRUD y G2_SRS29148_30719
Propósito	Guiar la transición desde el análisis funcional al análisis estructural y dinámico, manteniendo coherencia estricta entre requisito, caso de uso, clase y secuencia mediante códigos originales.

1. Propósito de la guía

Esta guía complementa la guía inicial de casos de uso (Guía 1), transitando desde el análisis funcional preliminar hacia el modelado estructural y dinámico para el módulo de Gestión de Tienda del sistema SPR (Sistema de Pedidos y Reservas para Canchas Don Pepito). Su objetivo fundamental es demostrar cómo los requisitos funcionales de nivel cero y sus casos de uso asociados se realizan formalmente a través de tres modelos conectados en PlantUML: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis.

El énfasis metodológico no radica en codificar interfaces visuales definitivas ni persistencias de base de datos relacionales, sino en comprender con exactitud qué responsabilidad cumple cada elemento del modelo orientado a objetos (frontera, control, entidad y repositorio) y cómo se garantiza la trazabilidad bidireccional y sin ambigüedades desde el requisito funcional del SRS hasta la invocación interactiva de mensajes entre objetos.

2. Relación general entre los tres modelos

El análisis orientado a objetos articula tres perspectivas complementarias para descomponer el sistema sin introducir detalles prematuros de implementación tecnológica:

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el actor con el módulo de tienda?	Diagrama de casos de uso con códigos originales (GT-01, GT-02, GT-11, GT-12, GT-10) y matriz RF-CU.	Permite delimitar el alcance funcional, identificar actores y derivar clases de frontera, control y entidad.
Clases de análisis	¿Qué objetos colaboran para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos <<boundary>>, <<control>>, <<entity>> y <<repository>>.	Define los objetos, atributos y firmas de operaciones que interactuarán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo interactúan actor, frontera, control, entidad y repositorio en el tiempo?	Diagramas de secuencia de análisis por cada operación clave (Ingresar, Modificar, Buscar y Reporte).	Valida que el flujo dinámico sea coherente con los requisitos funcionales y prepara el diseño detallado.

3. Caso base actualizado: SPR — Gestión de Tienda (Productos y Pedidos)

El módulo de Gestión de Tienda es operado principalmente por el actor Administrador, quien requiere registrar, actualizar y buscar productos del catálogo comercial, así como emitir reportes periódicos de los pedidos generados en el complejo deportivo. La interfaz administrativa centraliza estas operaciones sobre los siguientes datos clave del dominio:

Dato	Tipo sugerido	Descripción	Validación mínima
idProducto	Texto corto / UUID	Identificador único asignado internamente al producto.	Obligatorio, no vacío y único en el sistema.
nombre	Texto	Denominación comercial del producto en el catálogo.	Obligatorio, longitud >= 3 caracteres, no repetido en la misma categoría (RF-007).
categoria	Texto / Ref	Clasificación organizativa del producto (Bebidas, Snacks, Accesorios).	Obligatoria; debe existir en la lista de categorías registradas (RF-013/014).
precio	Decimal	Precio unitario de venta al público en dólares estadounidenses.	Obligatorio, numérico estrictamente positivo con hasta dos decimales (RF-007).
stock	Entero	Cantidad de existencias disponibles para despacho.	Obligatorio, valor entero mayor o igual a cero (RF-007); modificable en RF-009.
filtrosReporte	Fechas, Estado, Vendedor	Criterios de delimitación de pedidos para generar el reporte.	Rango de fechas de hasta un mes (RF-044); estado y vendedor válidos (GT-02).

Las acciones observables en la interfaz administrativa se traducen en los siguientes casos de uso analizados:

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Ingresar Producto	GT-11 Ingresar Producto [RF-007]	Registra un nuevo producto tras validar obligatoriamente nombre, precio, categoría y stock inicial único.
Modificar Producto	GT-12 Modificar Producto [RF-009]	Actualiza el precio, stock, nombre o categoría de un producto existente previa búsqueda por ID.
Buscar Producto	GT-10 Buscar Producto [RF-010]	Consulta y recupera los productos del catálogo por criterio de nombre o categoría (máx. 8 por página).
Generar Reporte Pedidos	GT-02 Generar Reporte de Pedidos [RF-044]	Recupera y compila el historial consolidado de pedidos según rango de fecha, estado y vendedor para exportación.

4. Requisitos funcionales de nivel 0 y trazabilidad

Se conservan estrictamente los códigos originales, redacciones oficiales y prioridades definidos en el SRS (G2_SRS29148_30719), vinculándolos con la arquitectura de casos de uso:

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-007	Registrar producto. El sistema deberá permitir al administrador registrar productos con nombre, categoría, precio y stock inicial.	Administrador	GT-11 Ingresar Producto (alcance GT-01 Gestionar Tienda)
RF-009	Modificar producto. El sistema deberá permitir al administrador modificar el nombre, categoría, precio o cantidad de un producto registrado.	Administrador	GT-12 Modificar Producto (alcance GT-01 Gestionar Tienda)
RF-010	Consultar productos registrados. El sistema deberá permitir al administrador consultar productos mediante filtros de búsqueda (máx. 8 por pág.).	Administrador	GT-10 Buscar Producto (alcance GT-01 Gestionar Tienda)
RF-044	Generar reportes del sistema (módulo pedidos). El sistema deberá permitir filtrar por rango de fechas (hasta 1 mes) y exportar a PDF o Excel.	Administrador	GT-02 Generar Reporte de Pedidos

5. Diagrama de casos de uso de análisis

El diagrama representa las interacciones del Administrador con las funciones del módulo dentro del límite del sistema. Siguiendo el principio de trazabilidad con códigos originales, se visualiza cómo GT-01 Gestionar Tienda agrupa por alcance funcional a las operaciones de ingreso (GT-11), modificación (GT-12) y búsqueda (GT-10), complementado por el caso de reporte consolidado (GT-02). La validación de datos se modela formalmente como una relación <<include>> obligatoria ejecutada por el actor secundario Sistema antes de confirmar el registro o actualización.

5.1 Justificación técnica de trazabilidad y resolución de hallazgos del SRS

En coherencia con la Guía 1 y la auditoría de requisitos del SRS (G2_SRS29148_30719), se documentan dos precisiones arquitectónicas fundamentales:

- **1. Asociación directa en GT-02 vs. <<extend>> del SRS:** En el diagrama inicial del SRS v4.0, GT-02 figuraba con una relación <<extend>> hacia GT-01. Sin embargo, el análisis de las especificaciones de Casos de Uso Integrados demuestra que el Administrador accede a generar reportes de pedidos como una acción autónoma desde el menú del módulo, sin depender de un flujo base interrumpido ni de un punto de extensión condicional. Por ello, tanto en la Guía 1 como en esta Guía 2 se modela mediante asociación directa Administrador -- GT-02, eliminando el acoplamiento artificial.
- **2. Descomposición funcional controlada (N0 a N1):** Mientras que la Guía 1 mantuvo una vista puramente macroscópica de Nivel 0 (GT-01 y GT-02), la Guía 2 realiza la transición metodológica requerida hacia el análisis orientado a objetos (N0 -> N1). Se descompone el caso contenedor GT-01 en sus operaciones atómicas con código original (GT-11 Ingresar, GT-12 Modificar y GT-10 Buscar) y el caso reutilizado 'Validar datos del producto' (<<include>>). Esta descomposición es la que posibilita asignar responsabilidades unívocas a las clases BCE y construir diagramas de secuencia sin sobrecarga.

Código PlantUML sugerido:

```
@startuml
skinparam packageStyle rectangle
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 12
skinparam backgroundColor white
skinparam usecaseBackgroundColor #EAF2F8
skinparam usecaseBorderColor #234F73
left to right direction

actor "Administrador" as Admin
```

```
actor "Sistema" as Sistema

rectangle "SPR - Gestión de Tienda | N0_SPR / N1_SPR" {
  usecase "GT-11\nIngresar Producto\n[RF-007]" as GT11
  usecase "GT-12\nModificar Producto\n[RF-009]" as GT12
  usecase "GT-10\nBuscar Producto\n[RF-010]" as GT10
  usecase "GT-02\nGenerar Reporte de Pedidos\n[RF-044]" as GT02
  usecase "Validar datos\ndel producto" as UC_Val
}

Admin --> GT11
Admin --> GT12
Admin --> GT10
Admin --> GT02

GT11 ..> UC_Val : <<include>>
GT12 ..> UC_Val : <<include>>

note right of UC_Val
Validaciones obligatorias (RF-007, RF-009):
- Nombre obligatorio y único en su categoría
- Precio numérico decimal (> 0)
- Stock inicial entero (>= 0)
end note

note top of GT11
Operaciones derivadas
por alcance funcional de:
GT-01 Gestionar Tienda
end note

note bottom of GT02
Filtros RF-044 / GT-02:
- Rango fechas (hasta 1 mes)
- Estado de pedido y vendedor
- Exportable a PDF y Excel
end note
@enduml
```

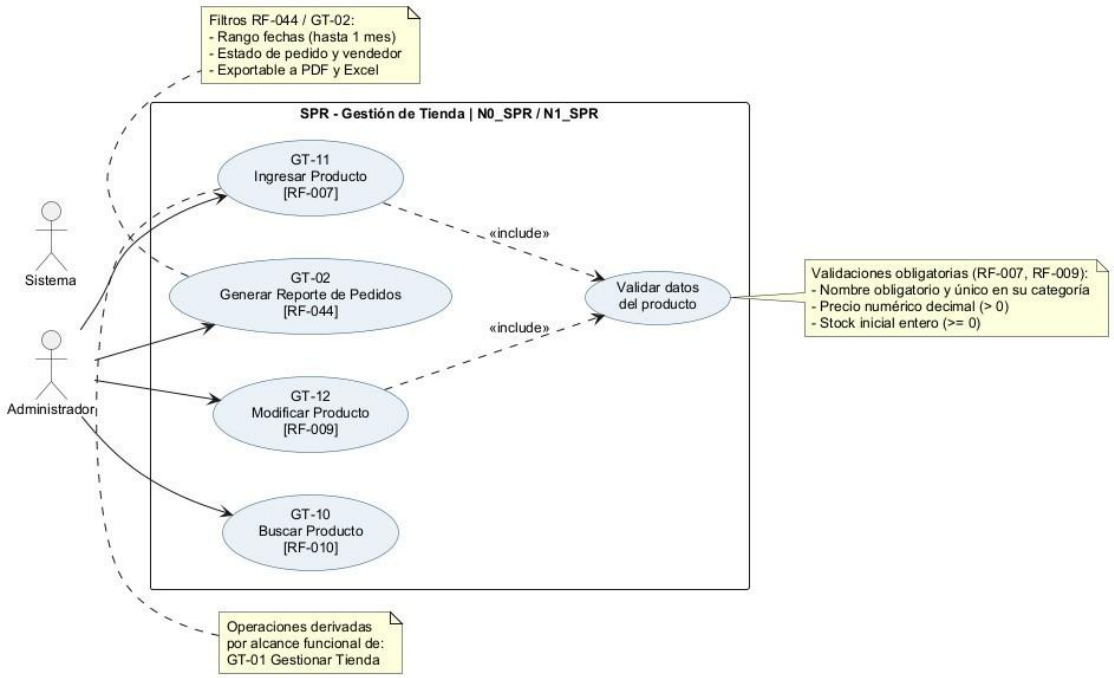


Figura: Diagrama de casos de uso de análisis para SPR — Gestión de Tienda

6. Derivación de clases de análisis

A partir de los casos de uso se identifican clases de análisis aplicando el patrón Boundary-Control-Entity (BCE) complementado con estereotipos Repository para representar la gestión de colecciones y persistencia sin acoplarse a detalles de infraestructura:

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con la Gestión de Tienda
Boundary / Frontera	Gestiona la interacción bidireccional con el Administrador: captura eventos y presenta datos.	PantallaGestionTienda	Representa el módulo visual con controles para ingresar, modificar, buscar productos y solicitar reportes.
Control	Coordina el flujo de control, aplica validaciones y orquesta operaciones entre frontera y entidades.	ControlGestionTienda	Valida reglas de negocio de productos, procesa transacciones de catálogo y solicita reportes de pedidos.
Entity / Entidad	Encapsula los atributos y el estado de la información persistente del negocio.	Producto y Pedido	Producto conserva id, nombre, categoría, precio y stock. Pedido encapsula id, fecha, estado, vendedor y total.
Repositorio / Colección	Representa la colección persistente de objetos del dominio en memoria o almacenamiento.	RepositorioProducto y RepositorioPedido	Permite consultar por filtros, verificar duplicidad, guardar, actualizar y listar productos y pedidos.

Código PlantUML del diagrama de clases de análisis:

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor white
left to right direction

class "PantallaGestionTienda" <<boundary>> {
  - txtIdProducto: String
  - txtNombre: String
  - txtCategoria: String
  - txtPrecio: double
  - txtStock: int
  - txtFiltroBusqueda: String
  - txtFiltroFecha: String
  - txtFiltroEstado: String
  - txtFiltroVendedor: String
  + clickIngresarProducto()
  + clickModificarProducto()
  + clickBuscarProducto()
  + clickGenerarReportePedidos()
  + mostrarMensaje(mensaje: String)
  + mostrarTablaProductos(productos: List<Producto>)
  + mostrarReportePedidos(pedidos: List<Pedido>)
}

class "ControlGestionTienda" <<control>> {
  + ingresarProducto(id: String, nombre: String, cat: String, precio: double, stock:
int): Resultado
  + modificarProducto(id: String, nombre: String, cat: String, precio: double, stock:
int): Resultado
  + buscarProducto(criterio: String): List<Producto>
  + generarReportePedidos(fecha: String, estado: String, vendedor: String): List<Pedido>
  + validarDatosProducto(nombre: String, precio: double, stock: int): boolean
}

class "Producto" <<entity>> {
  - idProducto: String
  - nombre: String
  - categoria: String
  - precio: double
  - stock: int
  + Producto(id: String, nombre: String, cat: String, precio: double, stock: int)
  + getIdProducto(): String
  + getNombre(): String
  + getCategoria(): String
  + getPrecio(): double
  + getStock(): int
  + setPrecio(precio: double): void
  + setStock(stock: int): void
}

class "Pedido" <<entity>> {
  - idPedido: String
  - fecha: String
  - estado: String
  - vendedor: String
  - total: double
  + Pedido(id: String, fecha: String, estado: String, vendedor: String, total: double)
  + getIdPedido(): String
  + getFecha(): String
  + getEstado(): String
```

```
  + getVendedor(): String
  + getTotal(): double
}

class "RepositorioProducto" <<repository>> {
  + existeId(id: String): boolean
  + existeNombreEnCategoria(nombre: String, cat: String): boolean
  + guardar(producto: Producto): void
  + buscarPorId(id: String): Producto
  + buscarPorNombreOCategoria(filtro: String): List<Producto>
  + actualizar(producto: Producto): void
}

class "RepositorioPedido" <<repository>> {
  + filtrarPedidos(fecha: String, estado: String, vendedor: String): List<Pedido>
}

PantallaGestionTienda --> ControlGestionTienda : solicita operación
ControlGestionTienda --> Producto : crea / actualiza
ControlGestionTienda --> Pedido : consulta datos
ControlGestionTienda --> RepositorioProducto : gestiona persistencia
ControlGestionTienda --> RepositorioPedido : consulta pedidos
RepositorioProducto o-- "0..*" Producto : almacena
RepositorioPedido o-- "0..*" Pedido : contiene
@enduml
```

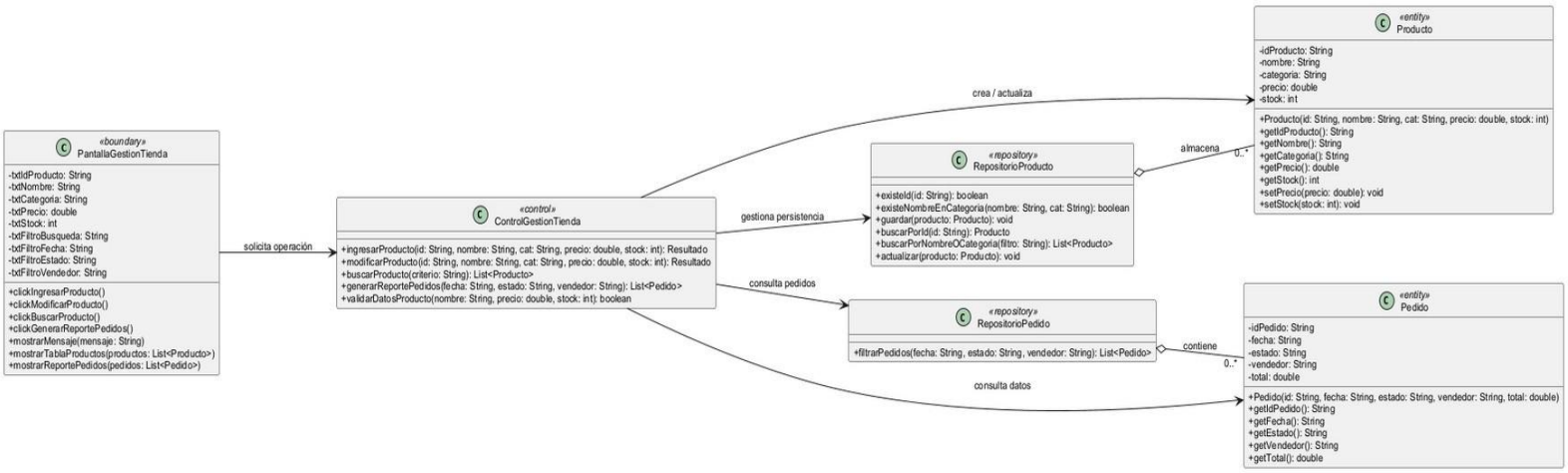


Figura: Diagrama de clases de análisis para SPR — Gestión de Tienda

7. Matriz de relación: casos de uso, clases y secuencias

La matriz de trazabilidad estructural-dinámica demuestra cómo cada caso de uso se vincula rigurosamente con los objetos BCE y su interacción temporal esperada:

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
GT-11 Ingresar Producto [RF-007]	PantallaGestionTiend a (captura datos y botón)	ControlGestionTienda (valida y orquesta)	Instancia Producto y almacena en RepositorioProducto	Secuencia: ingresar datos -> validar datos -> verificar unicidad -> instanciar Producto -> guardar en repositorio -> confirmar.
GT-12 Modificar Producto [RF-009]	PantallaGestionTiend a (captura ID y cambios)	ControlGestionTiend a (valida y coordina)	Modifica Producto y persiste en RepositorioProducto	Secuencia: ingresar cambios -> validar -> buscar por ID -> actualizar atributos de Producto -> actualizar repositorio -> confirmar.
GT-10 Buscar Producto [RF-010]	PantallaGestionTiend a (criterio y tabla)	ControlGestionTiend a (gestiona consulta)	RepositorioProducto recupera colección de Producto	Secuencia: ingresar criterio -> buscar en repositorio -> retornar lista (máx. 8) -> presentar tabla de catálogo.
GT-02 Generar Reporte Pedidos [RF-044]	PantallaGestionTiend a (filtros y reporte)	ControlGestionTiend a (procesa solicitud)	RepositorioPedido filtra colección de Pedido	Secuencia: seleccionar filtros -> consultar pedidos a RepositorioPedido -> recibir lista -> desplegar reporte y exportar.

8. Diagramas de secuencia de análisis

Los diagramas de secuencia ilustran la colaboración temporal de los objetos de análisis para satisfacer los flujos de trabajo del módulo. Se elabora un diagrama individualizado para cada operación clave garantizando máxima legibilidad técnica y trazabilidad de los códigos originales.

8.1 Secuencia: GT-11 Ingresar Producto [RF-007]

Modela el registro de un producto, verificando precondiciones de validación de campos obligatorios y unicidad de nombre en la categoría antes de crear y almacenar la entidad.

```
@startuml
title Secuencia de análisis - GT-11 Ingresar Producto [RF-007]
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor white

actor "Administrador" as Admin
boundary "PantallaGestionTienda" as UI
control "ControlGestionTienda" as Ctrl
entity "Producto" as Prod
database "RepositorioProducto" as Repo

Admin -> UI : Ingresa datos (nombre, categoría, precio, stock)
Admin -> UI : Clic en Ingresar Producto
UI -> Ctrl : ingresarProducto(id, nombre, cat, precio, stock)
Ctrl -> Ctrl : validarDatosProducto(nombre, precio, stock)
Ctrl -> Repo : existeNombreEnCategoria(nombre, cat)
Repo --> Ctrl : false
Ctrl -> Prod : new Producto(id, nombre, cat, precio, stock)
Ctrl -> Repo : guardar(Prod)
Repo --> Ctrl : guardado exitoso
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y catálogo actualizado
@enduml
```

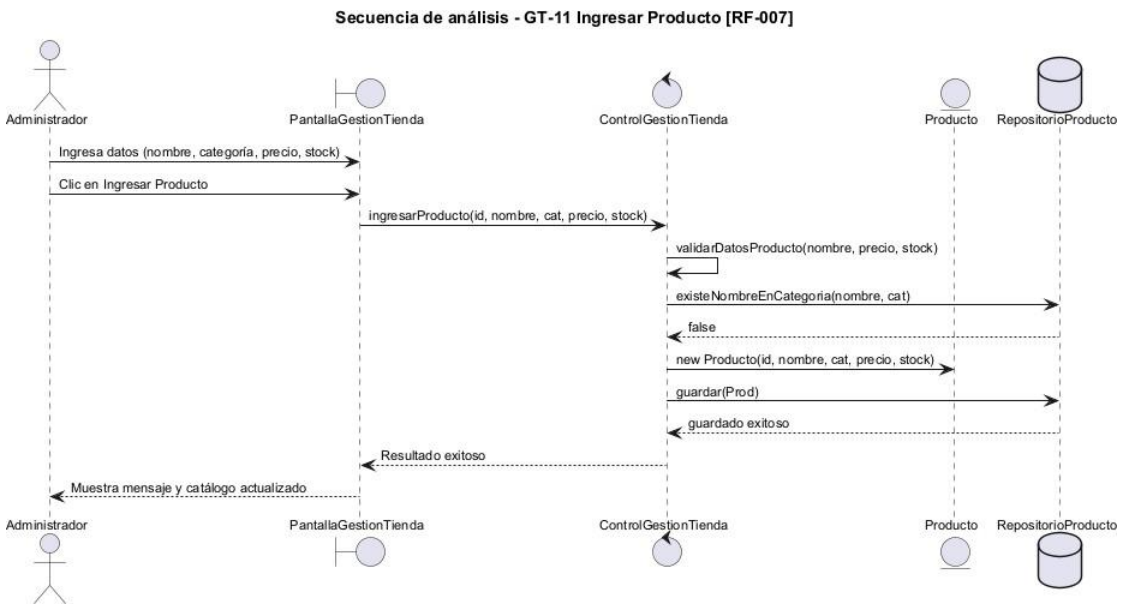


Figura: Secuencia de análisis — GT-11 Ingresar Producto [RF-007]

8.2 Secuencia: GT-12 Modificar Producto [RF-009]

Modela la actualización de datos de un producto (precio, stock, categoría), asegurando que el producto exista previamente en el repositorio antes de aplicar los cambios en un plazo menor a 5 segundos.

```
@startuml
title Secuencia de análisis - GT-12 Modificar Producto [RF-009]
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor white

actor "Administrador" as Admin
boundary "PantallaGestionTienda" as UI
control "ControlGestionTienda" as Ctrl
database "RepositorioProducto" as Repo
entity "Producto" as Prod

Admin -> UI : Ingresa ID y nuevos datos (precio, stock)
Admin -> UI : Clic en Modificar Producto
UI -> Ctrl : modificarProducto(id, nombre, cat, precio, stock)
Ctrl -> Ctrl : validarDatosProducto(nombre, precio, stock)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Producto encontrado
Ctrl -> Prod : setPrecio(precio) / setStock(stock)
Ctrl -> Repo : actualizar(Prod)
Repo --> Ctrl : actualización exitosa
Ctrl --> UI : Resultado exitoso (visible < 5s)
UI --> Admin : Muestra confirmación y catálogo actualizado
@enduml
```

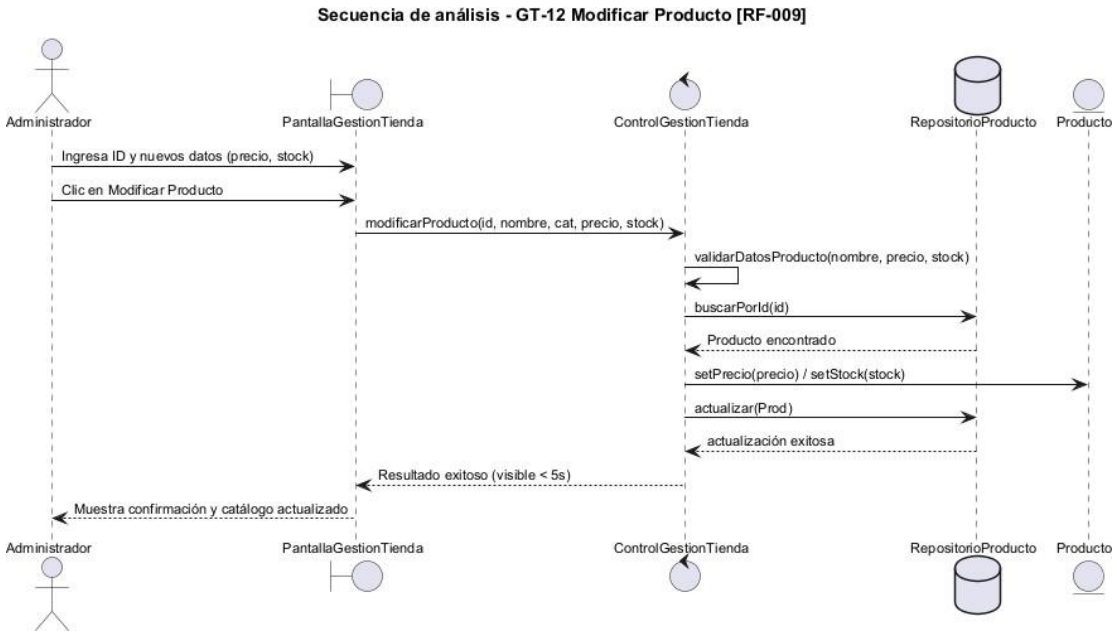


Figura: Secuencia de análisis — GT-12 Modificar Producto [RF-009]

8.3 Secuencia: GT-10 Buscar Producto [RF-010]

Modela la consulta de productos por criterio de búsqueda, retornando una vista paginada (máximo 8 registros) en un tiempo de respuesta inferior a 3 segundos.

```
@startuml
title Secuencia de análisis - GT-10 Buscar Producto [RF-010]
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor white

actor "Administrador" as Admin
boundary "PantallaGestionTienda" as UI
control "ControlGestionTienda" as Ctrl
database "RepositorioProducto" as Repo

Admin -> UI : Ingresa criterio de búsqueda (nombre o categoría)
Admin -> UI : Clic en Buscar Producto
UI -> Ctrl : buscarProducto(criterio)
Ctrl -> Repo : buscarPorNombreOCategoria(criterio)
Repo --> Ctrl : listaProductos (máx. 8 por pág.)
Ctrl --> UI : listaProductos
UI --> Admin : Muestra tabla con productos encontrados (< 3s)
@enduml
```

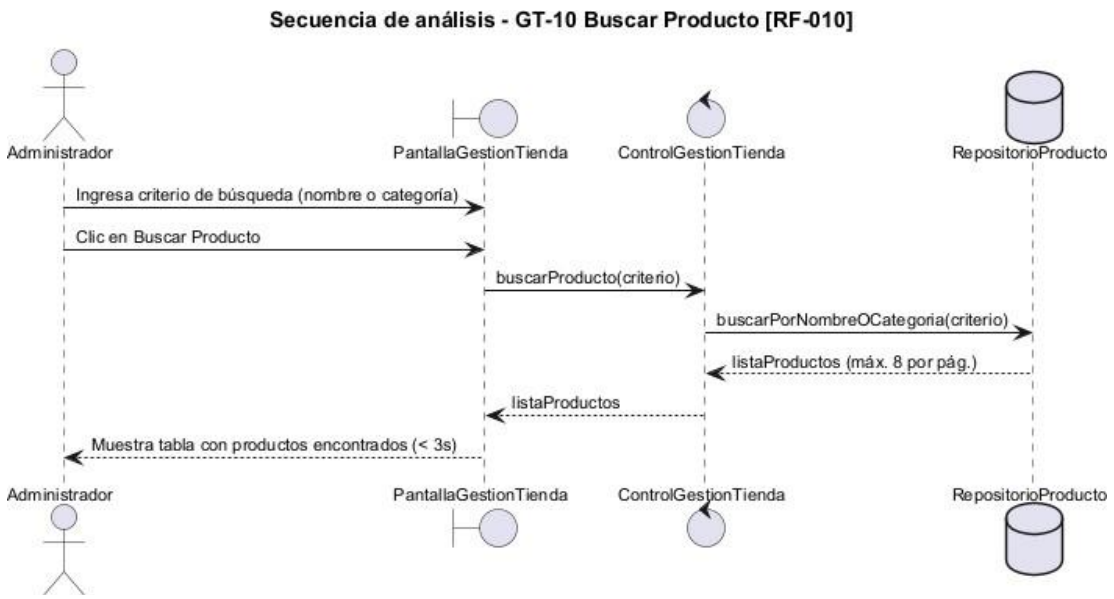


Figura: Secuencia de análisis — GT-10 Buscar Producto [RF-010]

8.4 Secuencia: GT-02 Generar Reporte de Pedidos [RF-044]

Modela la compilación de pedidos históricos basada en filtros de fecha (hasta 1 mes), estado y vendedor, desplegando el consolidado en pantalla y habilitando su exportación formal a PDF o Excel.

```
@startuml
title Secuencia de análisis - GT-02 Generar Reporte de Pedidos [RF-044]
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor white

actor "Administrador" as Admin
boundary "PantallaGestionTienda" as UI
control "ControlGestionTienda" as Ctrl
database "RepositorioPedido" as RepoPed

Admin -> UI : Selecciona filtros (fechas, estado, vendedor)
Admin -> UI : Clic en Generar Reporte
UI -> Ctrl : generarReportePedidos(fecha, estado, vendedor)
Ctrl -> RepoPed : filtrarPedidos(fecha, estado, vendedor)
RepoPed --> Ctrl : listaPedidos
Ctrl --> UI : listaPedidos
UI --> Admin : Despliega reporte de pedidos (< 5s) y opción PDF/Excel
@enduml
```

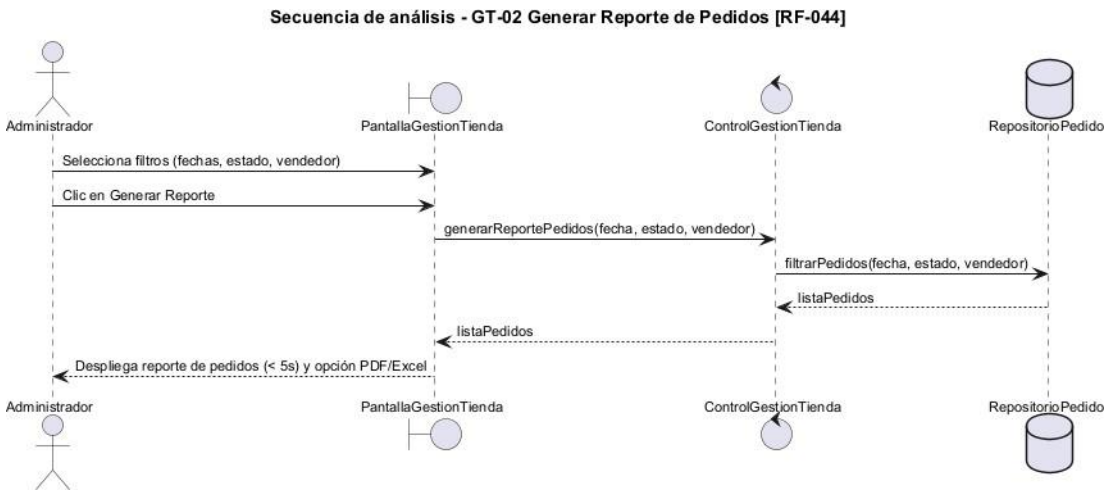


Figura: Secuencia de análisis — GT-02 Generar Reporte de Pedidos [RF-044]

9. Especificación breve de casos de uso conectada con las secuencias

La especificación textual resume formalmente el comportamiento esperado en coherencia directa con los flujos de las secuencias modeladas:

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
GT-11 Ingresar Producto [RF-007]	Administrador autenticado con rol administrativo. Nombre no duplicado en categoría.	Ingresar datos -> validar reglas de negocio -> verificar unicidad -> instanciar entidad -> persistir en repositorio.	Producto registrado en el catálogo y visible de inmediato.
GT-12 Modificar Producto [RF-009]	El producto a modificar debe existir registrado en el catálogo.	Ingresar ID y modificaciones -> validar campos -> buscar producto -> actualizar valores -> persistir cambios.	Datos del producto actualizados en catálogo en menos de 5 segundos.
GT-10 Buscar Producto [RF-010]	Catálogo con productos registrados disponibles.	Ingresar criterio (nombre/categoría) -> consultar repositorio -> recuperar colección -> presentar tabla paginada.	Lista de hasta 8 productos por página desplegada en menos de 3 segundos.
GT-02 Generar Reporte Pedidos [RF-044]	Debe existir al menos un pedido registrado en el periodo seleccionado.	Definir filtros (fecha <= 1 mes, estado, vendedor) -> solicitar compilación -> filtrar pedidos -> presentar y habilitar exportación.	Reporte estructurado generado en menos de 5 segundos con opción PDF/Excel.

10. Procedimiento recomendado para el estudiante

Para asegurar la integridad analítica y la total trazabilidad entre artefactos, se recomienda seguir el siguiente proceso metodológico:

- **1.** Revisar el SRS del proyecto e identificar los requisitos funcionales seleccionados conservando códigos originales (RF-007, RF-009, RF-010, RF-044).
- **2.** Delimitar el alcance modular en casos de uso de nivel cero (GT-01, GT-02) y especificar las operaciones de análisis derivadas (GT-11, GT-12, GT-10).
- **3.** Modelar el diagrama de casos de uso en PlantUML, justificando las relaciones <<include>> indispensables (como la validación previa de datos).
- **4.** Derivar las clases de análisis aplicando estereotipos BCE (boundary, control, entity) y repository, definiendo métodos y tipos de datos precisos.
- **5.** Construir un diagrama de secuencia de análisis por cada caso de uso principal, asegurando orden temporal y coincidencia exacta con los métodos de clase.
- **6.** Verificar la consistencia semántica: cada mensaje enviado en una secuencia debe corresponder a una firma declarada en el diagrama de clases.
- **7.** Elaborar las matrices de trazabilidad para auditar que ningún requisito o caso de uso quede sin realización estructural ni dinámica.

11. Lista de verificación de coherencia

- **[X]** Cada requisito funcional seleccionado cuenta con al menos un caso de uso asociado y trazado con su código original.
- **[X]** Cada caso de uso modelado posee su correspondiente diagrama de secuencia de análisis estructurado.
- **[X]** Todos los objetos y participantes de los diagramas de secuencia existen en el diagrama de clases de análisis.
- **[X]** Cada mensaje y signatura invocada en las secuencias coincide exactamente con los métodos definidos en las clases.
- **[X]** Las entidades Producto y Pedido conservan todos los atributos requeridos por las reglas de negocio del SRS.
- **[X]** La validación obligatoria de datos se ejecuta rigurosamente antes de persistir cualquier registro o modificación.
- **[X]** Los diagramas de análisis no mezclan componentes tecnológicos de frameworks ni sentencias SQL concretas.

12. Rúbrica breve de evaluación sobre 20 puntos

La evaluación académica del entregable se rige bajo la rúbrica oficial estandarizada:

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad entre modelos	Conecta rigurosamente RF, casos de uso originales, clases BCE y diagramas de secuencia.	5 pts	Matriz de relación completa y coherente (Sección 7).
Diagrama de casos de uso	Representa actor, límites, casos originales y relaciones <<include>> debidamente justificadas.	4 pts	Código y diagrama PlantUML compilado (Sección 5).
Diagrama de clases de análisis	Identifica boundary, control, entity y repository con atributos y firmas de métodos precisos.	4 pts	Código y diagrama PlantUML de clases (Sección 6).

Diagramas de secuencia	Modela flujos coherentes y completos para cada una de las cuatro operaciones del módulo.	5 pts	Cuatro secuencias PlantUML funcionales (Sección 8).
Presentación técnica	Documento riguroso, ordenado, ortografía impecable y códigos PlantUML sin errores sintácticos.	2 pts	Documentación formal y entrega completa.

13. Formato mínimo de entrega sugerido

Estructura requerida para la entrega formal de la práctica de laboratorio:

Sección	Contenido mínimo	Evidencia
Portada y Metadatos	Datos institucionales, asignatura, NRC, integrantes del equipo, proyecto y fecha.	Página inicial formal.
Casos de Uso de Análisis	Requisitos funcionales del SRS, diagrama PlantUML de casos de uso y notas de trazabilidad.	Código PlantUML + captura renderizada.
Clases de Análisis	Diagrama con estereotipos boundary, control, entity y repository con métodos.	Código PlantUML + captura renderizada.
Secuencias de Análisis	Una secuencia por cada caso de uso clave (Ingresar, Modificar, Buscar y Reporte).	4 códigos PlantUML + 4 capturas.
Trazabilidad Integral	Matriz que articula RF, casos de uso, clases BCE y secuencias.	Tabla de relación consolidada.
Conclusión Técnica	Reflexión metodológica sobre la coherencia y evolución del modelo.	Texto formal de cierre.

14. Conclusión técnica

La articulación progresiva entre casos de uso de análisis, clases de análisis y diagramas de secuencia permite verificar que el modelo del módulo de Gestión de Tienda no se ha construido como artefactos aislados. Los casos de uso de nivel cero (GT-01 y GT-02) definen las intenciones del Administrador dentro del alcance del sistema; las clases de análisis especifican la estructura de colaboración segregando la interacción (PantallaTienda, PantallaReportePedidos), la coordinación lógica (ControlProducto, ControlReportePedidos), la abstracción del dominio (Producto, Pedido) y la gestión de almacenamiento (RepositorioProducto, RepositorioPedido); finalmente, los diagramas de secuencia demuestran el flujo dinámico ordenado en el tiempo y validan que cada interacción invoca métodos formalmente declarados. Esta coherencia estructural garantiza la trazabilidad exigida por la ingeniería de software y deja completamente preparado el sistema para las fases subsiguientes de diseño detallado y construcción.

La utilización sistemática de códigos originales (RF-007, RF-009, RF-010, RF-044, GT-01, GT-02, GT-11, GT-12, GT-10) asegura una trazabilidad impecable hacia adelante y hacia atrás, lo cual mitiga riesgos de dispersión en el alcance, facilita auditorías de calidad de software y establece una base robusta e inequívoca para la posterior etapa de diseño arquitectónico y construcción de software.

15. Fuentes internas de referencia

- 01_Guia_PlantUML_CU_Analisis_CRUD_Estudiante.pdf — Guía docente base de modelado de casos de uso de análisis en PlantUML.
- 02_Guia_2_casos_de_uso_clases_y_secuencia_CRUD.docx / .pdf — Guía docente 2 sobre relación entre casos de uso, clases y secuencias de análisis.
- G2_SRS29148_30719.docx / .pdf — Especificación de Requisitos de Software (SRS) del Sistema de Pedidos y Reservas para Canchas Don Pepito (SPR).
- Casos_de_Uso_Integrados.docx / .pdf — Catálogo integrado de especificaciones de casos de uso de nivel 0, nivel 1 y nivel 2 del proyecto SPR.
- Guia_PlantUML_SPR_Gestion_Tienda.docx — Guía 1 de desarrollo en PlantUML del módulo Gestión de Tienda elaborada por el Grupo 7.